Introduction to Theoretical Computer Science

CS2490 - Session 12: From REs to Automata

Pierre Cagne

Fall 2026

September 14, 2026

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. No pattern at all!

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern
- For each $a \in \Sigma$, a is a RE. — Patterns of length 1

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern
- For each $a \in \Sigma$, a is a RE. — Patterns of length 1
- If r_1 and r_2 are REs, then so are:

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern
- For each $a \in \Sigma$, a is a RE. — Patterns of length 1
- If r_1 and r_2 are REs, then so are:
  - $(r_1 + r_2)$ — Either ... or ...

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern
- For each $a \in \Sigma$, a is a RE. — Patterns of length 1
- If r_1 and r_2 are REs, then so are:
  - $(r_1 + r_2)$ — Either ... or ...
  - $(r_1 r_2)$ — Followed by ...

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern
- For each $a \in \Sigma$, a is a RE. — Patterns of length 1
- If r_1 and r_2 are REs, then so are:
  - $(r_1 + r_2)$ — Either ... or ...
  - $(r_1 r_2)$ — Followed by ...
  - $(r_1{}^*)$ — A sequence of ...

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern
- For each $a \in \Sigma$, a is a RE. — Patterns of length 1
- If r_1 and r_2 are REs, then so are:
  - $(r_1 + r_2)$ — Either ... or ...
  - $(r_1 r_2)$ — Followed by ...
  - $(r_1{}^*)$ — A sequence of ...

Omit parentheses when there is no ambiguity: * has the highest precedence, then concatenation, then $+$.

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern
- For each $a \in \Sigma$, a is a RE. — Patterns of length 1
- If r_1 and r_2 are REs, then so are:
  - $(r_1 + r_2)$ — Either ... or ...
  - $(r_1 r_2)$ — Followed by ...
  - $(r_1{}^*)$ — A sequence of ...

Omit parentheses when there is no ambiguity: * has the highest precedence, then concatenation, then $+$.

We write $\mathrm{L}(r)$ for the language described/denoted by the RE r.

Examples

Examples

- What is the language denoted by $(0 + 1)^*$?

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?
  All words over $\{a, b\}$ containing the substring bab.

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?
  All words over $\{a, b\}$ containing the substring bab.
- What is the language denoted by $(a + b)(a + b)^*$?

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?
  All words over $\{a, b\}$ containing the substring bab.
- What is the language denoted by $(a + b)(a + b)^*$?
  All non-empty words over $\{a, b\}$.

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?
  All words over $\{a, b\}$ containing the substring bab.
- What is the language denoted by $(a + b)(a + b)^*$?
  All non-empty words over $\{a, b\}$. I.e., L_1.

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?
  All words over $\{a, b\}$ containing the substring bab.
- What is the language denoted by $(a + b)(a + b)^*$?
  All non-empty words over $\{a, b\}$. I.e., L_1. Or L_2.

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?
  All words over $\{a, b\}$ containing the substring bab.
- What is the language denoted by $(a + b)(a + b)^*$?
  All non-empty words over $\{a, b\}$. I.e., L_1. Or L_2.
- What is the language denoted by $a(a + b)^* + b(a + b)^*$?

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?
  All words over $\{a, b\}$ *containing the substring bab.*
- What is the language denoted by $(a + b)(a + b)^*$?
  All non-empty words over $\{a, b\}$. *I.e.,* L_1. *Or* L_2.
- What is the language denoted by $a(a + b)^* + b(a + b)^*$?
  All non-empty words over $\{a, b\}$.

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?
  All words over $\{a, b\}$ containing the substring bab.
- What is the language denoted by $(a + b)(a + b)^*$?
  All non-empty words over $\{a, b\}$. I.e., L_1. Or L_2.
- What is the language denoted by $a(a + b)^* + b(a + b)^*$?
  All non-empty words over $\{a, b\}$. I.e., L_1/L_2 again.

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?
  All words over $\{a, b\}$ containing the substring bab.
- What is the language denoted by $(a + b)(a + b)^*$?
  All non-empty words over $\{a, b\}$. I.e., L_1. Or L_2.
- What is the language denoted by $a(a + b)^* + b(a + b)^*$?
  All non-empty words over $\{a, b\}$. I.e., L_1/L_2 again.

As for DFA/NFA, multiple REs can denote the same language.

Relation to DFA/NFA

Regular expressions are related to regular languages.

Relation to DFA/NFA

Regular expressions are related to regular languages.

Theorem

Let Σ be an alphabet. A language over Σ is regular if and only if there exists a RE that denotes it.

Relation to DFA/NFA

Regular expressions are related to regular languages.

Theorem
Let Σ be an alphabet. A language over Σ is regular if and only if there exists a RE that denotes it.

Proof.
($\Leftarrow$) From a RE r, construct a NFA $\mathcal{A}_r$ that recognizes $\mathrm{L}(r)$.

Relation to DFA/NFA

Regular expressions are related to regular languages.

Theorem

Let Σ be an alphabet. A language over Σ is regular if and only if there exists a RE that denotes it.

Proof.

($\Leftarrow$) From a RE r, construct a NFA $\mathcal{A}_r$ that recognizes $\mathrm{L}(r)$.

($\Rightarrow$) From a DFA/NFA $\mathcal{A}$, construct a RE $r_{\mathcal{A}}$ that denotes $\mathrm{L}(\mathcal{A})$.

□

Relation to DFA/NFA

Regular expressions are related to regular languages.

Theorem

Let Σ be an alphabet. A language over Σ is regular if and only if there exists a RE that denotes it.

Proof.

($\Leftarrow$) From a RE r, construct a NFA $\mathcal{A}_r$ that recognizes $\mathrm{L}(r)$. Today.

($\Rightarrow$) From a DFA/NFA $\mathcal{A}$, construct a RE $r_{\mathcal{A}}$ that denotes $\mathrm{L}(\mathcal{A})$.

□

Relation to DFA/NFA

Regular expressions are related to regular languages.

Theorem

Let Σ be an alphabet. A language over Σ is regular if and only if there exists a RE that denotes it.

Proof.

($\Leftarrow$) From a RE r, construct a NFA $\mathcal{A}_r$ that recognizes $\mathrm{L}(r)$. Today.

($\Rightarrow$) From a DFA/NFA $\mathcal{A}$, construct a RE $r_{\mathcal{A}}$ that denotes $\mathrm{L}(\mathcal{A})$. Next time.

□

RE to NFA: a little algorithm

To construct a NFA $\mathcal{A}_r$ from a RE r, we proceed *inductively* on the structure of r:

RE to NFA: a little algorithm

To construct a NFA $\mathcal{A}_r$ from a RE r, we proceed *inductively* on the structure of r:

1. First construct $\mathcal{A}_r$ for each of the base cases ($\varnothing$, ε, and $a \in \Sigma$).

RE to NFA: a little algorithm

To construct a NFA $\mathcal{A}_r$ from a RE r, we proceed *inductively* on the structure of r:

1. First construct $\mathcal{A}_r$ for each of the base cases ($\varnothing$, ε, and $a \in \Sigma$).
2. Then construct $\mathcal{A}_r$ for each of the inductive cases ($r_1 + r_2$, $r_1 r_2$, and ${r_1}^*$).

RE to NFA: a little algorithm

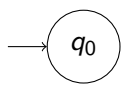
To construct a NFA $\mathcal{A}_r$ from a RE r, we proceed *inductively* on the structure of r:

1. First construct $\mathcal{A}_r$ for each of the base cases ($\varnothing$, ε, and $a \in \Sigma$).
2. Then construct $\mathcal{A}_r$ for each of the inductive cases ($r_1 + r_2$, $r_1 r_2$, and ${r_1}^*$).

RE to NFA: a little algorithm

To construct a NFA $\mathcal{A}_r$ from a RE r, we proceed *inductively* on the structure of r:

1. First construct $\mathcal{A}_r$ for each of the base cases ($\varnothing$, ε, and $a \in \Sigma$).
2. Then construct $\mathcal{A}_r$ for each of the inductive cases ($r_1 + r_2$, $r_1 r_2$, and ${r_1}^*$).

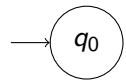
- $\mathcal{A}_\varnothing$: 

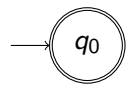
$\mathcal{A}_\varnothing$ recognizes $\emptyset$.

RE to NFA: a little algorithm

To construct a NFA $\mathcal{A}_r$ from a RE r, we proceed *inductively* on the structure of r:

1. First construct $\mathcal{A}_r$ for each of the base cases ($\varnothing$, ε, and $a \in \Sigma$).
2. Then construct $\mathcal{A}_r$ for each of the inductive cases ($r_1 + r_2$, $r_1 r_2$, and ${r_1}^*$).

- $\mathcal{A}_\varnothing$: 

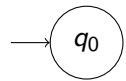
  $\mathcal{A}_\varnothing$ recognizes $\emptyset$.
- $\mathcal{A}_\varepsilon$: 

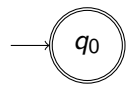
  $\mathcal{A}_\varepsilon$ recognizes ε.

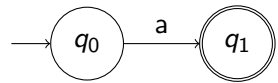
RE to NFA: a little algorithm

To construct a NFA $\mathcal{A}_r$ from a RE r, we proceed *inductively* on the structure of r:

1. First construct $\mathcal{A}_r$ for each of the base cases ($\varnothing$, ε, and $a \in \Sigma$).
2. Then construct $\mathcal{A}_r$ for each of the inductive cases ($r_1 + r_2$, $r_1 r_2$, and ${r_1}^*$).

- $\mathcal{A}_\varnothing$: 

$\mathcal{A}_\varnothing$ recognizes $\emptyset$.
- $\mathcal{A}_\varepsilon$: 

$\mathcal{A}_\varepsilon$ recognizes ε.
- $\mathcal{A}_a$: 

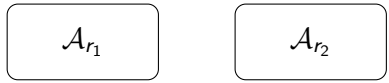
$\mathcal{A}_a$ recognizes $\{a\}$.

RE to NFA: union

Suppose $r = r_1 + r_2$ and we have already constructed $\mathcal{A}_{r_1}$ and $\mathcal{A}_{r_2}$.

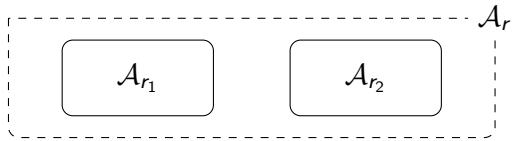
RE to NFA: union

Suppose $r = r_1 + r_2$ and we have already constructed $\mathcal{A}_{r_1}$ and $\mathcal{A}_{r_2}$.



RE to NFA: union

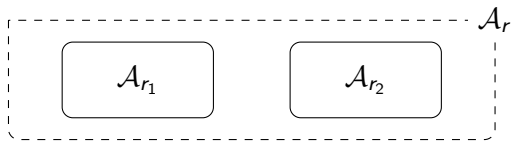
Suppose $r = r_1 + r_2$ and we have already constructed $\mathcal{A}_{r_1}$ and $\mathcal{A}_{r_2}$.



To construct $\mathcal{A}_r$: just consider $\mathcal{A}_{r_1}$ and $\mathcal{A}_{r_2}$ together.

RE to NFA: union

Suppose $r = r_1 + r_2$ and we have already constructed $\mathcal{A}_{r_1}$ and $\mathcal{A}_{r_2}$.

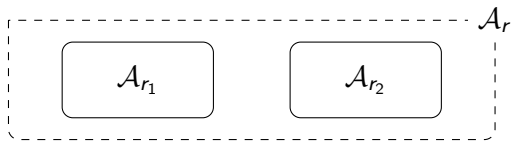


To construct $\mathcal{A}_r$: just consider $\mathcal{A}_{r_1}$ and $\mathcal{A}_{r_2}$ together.

(Note: this applies to any $\mathcal{A}$ and $\mathcal{B}$ to produce a NFA recognizing $\mathrm{L}(\mathcal{A}) \cup \mathrm{L}(\mathcal{B})$.)

RE to NFA: union

Suppose $r = r_1 + r_2$ and we have already constructed $\mathcal{A}_{r_1}$ and $\mathcal{A}_{r_2}$.



To construct $\mathcal{A}_r$: just consider $\mathcal{A}_{r_1}$ and $\mathcal{A}_{r_2}$ together.

(Note: this applies to any $\mathcal{A}$ and $\mathcal{B}$ to produce a NFA recognizing $\mathsf{L}(\mathcal{A}) \cup \mathsf{L}(\mathcal{B})$.)

Formally: if we write $\mathcal{A}_{r_i} = (\Sigma, Q_i, \Delta_i, I_i, F_i)$ for $i = 1, 2$, then $\mathcal{A}_r = (\Sigma, Q, \Delta, I, F)$ where:

- $Q = Q_1 \cup Q_2$ (assume w.l.o.g. that Q_1 and Q_2 are disjoint),
- $I = I_1 \cup I_2$,
- $F = F_1 \cup F_2$,
- $\Delta(a, q) = \Delta_1(a, q)$ if $q \in Q_1$, and $\Delta(a, q) = \Delta_2(a, q)$ if $q \in Q_2$.

RE to NFA: concatenation

Suppose $r = r_1 \cdot r_2$ and we have already constructed $\mathcal{A}_{r_1}$ and $\mathcal{A}_{r_2}$.

RE to NFA: concatenation

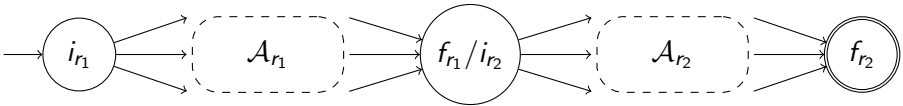
Suppose $r = r_1 \cdot r_2$ and we have already constructed $\mathcal{A}_{r_1}$ and $\mathcal{A}_{r_2}$.

Conceptually: want to "glue" the final state of $\mathcal{A}_{r_1}$ onto the initial state of $\mathcal{A}_{r_2}$, so that a run of $\mathcal{A}_{r_1}$ can be continued by a run of $\mathcal{A}_{r_2}$.

RE to NFA: concatenation

Suppose $r = r_1 \cdot r_2$ and we have already constructed $\mathcal{A}_{r_1}$ and $\mathcal{A}_{r_2}$.

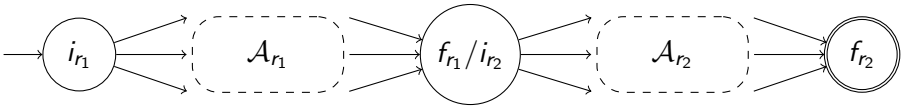
Conceptually: want to "glue" the final state of $\mathcal{A}_{r_1}$ onto the initial state of $\mathcal{A}_{r_2}$, so that a run of $\mathcal{A}_{r_1}$ can be continued by a run of $\mathcal{A}_{r_2}$.



RE to NFA: concatenation

Suppose $r = r_1 \cdot r_2$ and we have already constructed $\mathcal{A}_{r_1}$ and $\mathcal{A}_{r_2}$.

Conceptually: want to "glue" the final state of $\mathcal{A}_{r_1}$ onto the initial state of $\mathcal{A}_{r_2}$, so that a run of $\mathcal{A}_{r_1}$ can be continued by a run of $\mathcal{A}_{r_2}$.

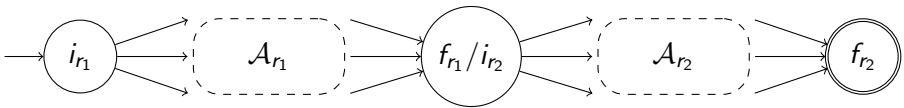


Intended runs of $\mathcal{A}_r$: a run of $\mathcal{A}_{r_1}$ from i_{r_1} to the glue state, then a run of $\mathcal{A}_{r_2}$ from the glue state to f_{r_2}.

RE to NFA: concatenation

Suppose $r = r_1 \cdot r_2$ and we have already constructed $\mathcal{A}_{r_1}$ and $\mathcal{A}_{r_2}$.

Conceptually: want to "glue" the final state of $\mathcal{A}_{r_1}$ onto the initial state of $\mathcal{A}_{r_2}$, so that a run of $\mathcal{A}_{r_1}$ can be continued by a run of $\mathcal{A}_{r_2}$.

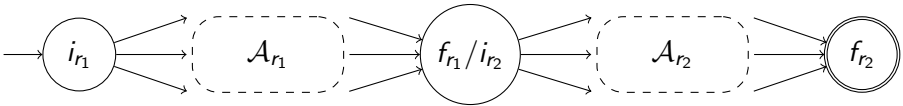


Intended runs of $\mathcal{A}_r$: a run of $\mathcal{A}_{r_1}$ from i_{r_1} to the glue state, then a run of $\mathcal{A}_{r_2}$ from the glue state to f_{r_2}. The glue state keeps the marks of f_{r_1} and i_{r_2} (initial if f_{r_1} was, final if i_{r_2} was).

RE to NFA: concatenation

Suppose $r = r_1 \cdot r_2$ and we have already constructed $\mathcal{A}_{r_1}$ and $\mathcal{A}_{r_2}$.

Conceptually: want to "glue" the final state of $\mathcal{A}_{r_1}$ onto the initial state of $\mathcal{A}_{r_2}$, so that a run of $\mathcal{A}_{r_1}$ can be continued by a run of $\mathcal{A}_{r_2}$.



Intended runs of $\mathcal{A}_r$: a run of $\mathcal{A}_{r_1}$ from i_{r_1} to the glue state, then a run of $\mathcal{A}_{r_2}$ from the glue state to f_{r_2}. The glue state keeps the marks of f_{r_1} and i_{r_2} (initial if f_{r_1} was, final if i_{r_2} was).

Are these the only runs?

RE to NFA: concatenation (cont'd)

Gluing is safe when:

RE to NFA: concatenation (cont'd)

Gluing is safe when:

1. $\mathcal{A}_{r_1}$ has exactly one final state f,

RE to NFA: concatenation (cont'd)

Gluing is safe when:

1. $\mathcal{A}_{r_1}$ has exactly one final state f,
2. $\mathcal{A}_{r_2}$ has exactly one initial state i,

RE to NFA: concatenation (cont'd)

Gluing is safe when:

1. $\mathcal{A}_{r_1}$ has exactly one final state f,
2. $\mathcal{A}_{r_2}$ has exactly one initial state i,
3. there is no transition out of f in $\mathcal{A}_{r_1}$, **or** no transition into i in $\mathcal{A}_{r_2}$.

RE to NFA: concatenation (cont'd)

Gluing is safe when:

1. $\mathcal{A}_{r_1}$ has exactly one final state f,
2. $\mathcal{A}_{r_2}$ has exactly one initial state i,
3. there is no transition out of f in $\mathcal{A}_{r_1}$, **or** no transition into i in $\mathcal{A}_{r_2}$.

Why 3.?

RE to NFA: concatenation (cont'd)

Gluing is safe when:

1. $\mathcal{A}_{r_1}$ has exactly one final state f,
2. $\mathcal{A}_{r_2}$ has exactly one initial state i,
3. there is no transition out of f in $\mathcal{A}_{r_1}$, **or** no transition into i in $\mathcal{A}_{r_2}$.

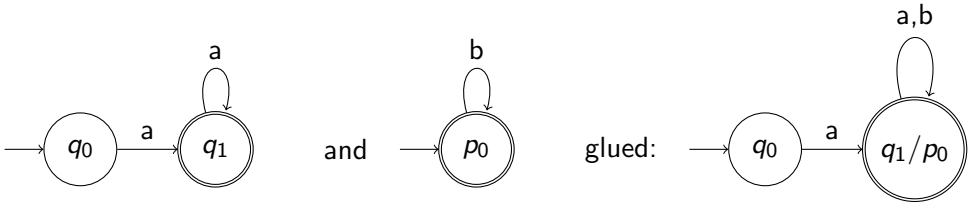
Why 3.? If there are both a transition into i and a transition out of f, a run can enter $\mathcal{A}_{r_2}$, come back to the glue state, and leave again through $\mathcal{A}_{r_1}$.

RE to NFA: concatenation (cont'd)

Gluing is safe when:

1. $\mathcal{A}_{r_1}$ has exactly one final state f,
2. $\mathcal{A}_{r_2}$ has exactly one initial state i,
3. there is no transition out of f in $\mathcal{A}_{r_1}$, **or** no transition into i in $\mathcal{A}_{r_2}$.

Why 3.? If there are both a transition into i and a transition out of f, a run can enter $\mathcal{A}_{r_2}$, come back to the glue state, and leave again through $\mathcal{A}_{r_1}$.

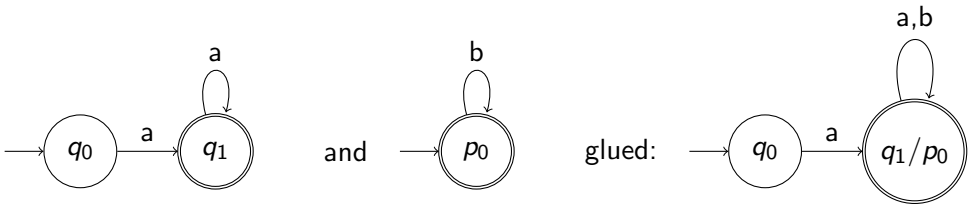


RE to NFA: concatenation (cont'd)

Gluing is safe when:

1. $\mathcal{A}_{r_1}$ has exactly one final state f,
2. $\mathcal{A}_{r_2}$ has exactly one initial state i,
3. there is no transition out of f in $\mathcal{A}_{r_1}$, **or** no transition into i in $\mathcal{A}_{r_2}$.

Why 3.? If there are both a transition into i and a transition out of f, a run can enter $\mathcal{A}_{r_2}$, come back to the glue state, and leave again through $\mathcal{A}_{r_1}$.



Gluing $\mathcal{A}_{aa^*}$ onto $\mathcal{A}_{b^*}$ accepts aba, which is not in $\mathrm{L}(aa^*b^*)$.

When gluing is not safe

If gluing is not safe, first normalize *one* of the two automata:

When gluing is not safe

If gluing is not safe, first normalize *one* of the two automata: $\mathcal{A}_{r_1}$ if 1. fails,

When gluing is not safe

If gluing is not safe, first normalize *one* of the two automata: $\mathcal{A}_{r_1}$ if 1. fails, $\mathcal{A}_{r_2}$ if 2. fails,

When gluing is not safe

If gluing is not safe, first normalize *one* of the two automata: $\mathcal{A}_{r_1}$ if 1. fails, $\mathcal{A}_{r_2}$ if 2. fails, either one if only 3. fails (a fresh f has no transition out of it, a fresh i none into it).

When gluing is not safe

If gluing is not safe, first normalize *one* of the two automata: $\mathcal{A}_{r_1}$ if 1. fails, $\mathcal{A}_{r_2}$ if 2. fails, either one if only 3. fails (a fresh f has no transition out of it, a fresh i none into it). Both only when 1. and 2. both fail.

When gluing is not safe

If gluing is not safe, first normalize *one* of the two automata: $\mathcal{A}_{r_1}$ if 1. fails, $\mathcal{A}_{r_2}$ if 2. fails, either one if only 3. fails (a fresh f has no transition out of it, a fresh i none into it). Both only when 1. and 2. both fail.

Definition

A NFA is normalized if it has:

- exactly one initial state, with no transitions *to* it,
- exactly one final state, different from the initial state, with no transitions *from* it.

When gluing is not safe

If gluing is not safe, first normalize *one* of the two automata: $\mathcal{A}_{r_1}$ if 1. fails, $\mathcal{A}_{r_2}$ if 2. fails, either one if only 3. fails (a fresh f has no transition out of it, a fresh i none into it). Both only when 1. and 2. both fail.

Definition

A NFA is normalized if it has:

- exactly one initial state, with no transitions *to* it,
- exactly one final state, different from the initial state, with no transitions *from* it.



When gluing is not safe

If gluing is not safe, first normalize *one* of the two automata: $\mathcal{A}_{r_1}$ if 1. fails, $\mathcal{A}_{r_2}$ if 2. fails, either one if only 3. fails (a fresh f has no transition out of it, a fresh i none into it). Both only when 1. and 2. both fail.

Definition

A NFA is normalized if it has:

- exactly one initial state, with no transitions *to* it,
- exactly one final state, different from the initial state, with no transitions *from* it.



A normalized NFA can be glued on either side.

Normalizing a NFA

Theorem

Any NFA $\mathcal{A}$ can be transformed into a normalized NFA $\mathcal{A}^{\mathrm{norm}}$ that recognizes $\mathsf{L}(\mathcal{A}) \setminus \{\varepsilon\}$.

Normalizing a NFA

Theorem

Any NFA $\mathcal{A}$ can be transformed into a normalized NFA $\mathcal{A}^{\mathrm{norm}}$ that recognizes $\mathrm{L}(\mathcal{A}) \setminus \{\varepsilon\}$.

Proof.

Write $\mathcal{A} = (\Sigma, Q, \Delta, I, F)$. Then construct $\mathcal{A}^{\mathrm{norm}} = (\Sigma, Q^{\mathrm{norm}}, \Delta^{\mathrm{norm}}, \{i\}, \{f\})$ as follows:

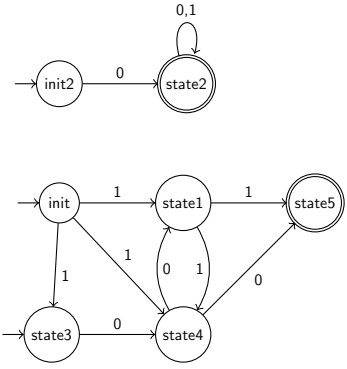
- $Q^{\mathrm{norm}} = Q \cup \{i, f\}$ where i and f are new states,
- $\Delta^{\mathrm{norm}}(a, q) = \Delta(a, q)$ for all $a \in \Sigma$ and $q \in Q$ when $\Delta(a, q) \cap F = \emptyset$,
- $\Delta^{\mathrm{norm}}(a, q) = \Delta(a, q) \cup \{f\}$ for all $a \in \Sigma$ and $q \in Q$ when $\Delta(a, q) \cap F \neq \emptyset$,
- $\Delta^{\mathrm{norm}}(a, i) = \bigcup_{q \in I} \Delta^{\mathrm{norm}}(a, q)$ for all $a \in \Sigma$,
- $\Delta^{\mathrm{norm}}(a, f) = \emptyset$ for all $a \in \Sigma$.



Normalizing a NFA

Theorem

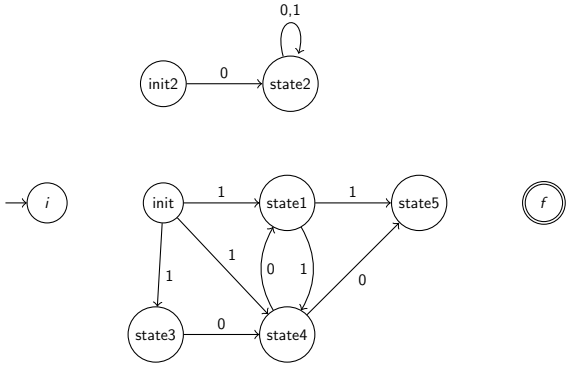
Any NFA $\mathcal{A}$ can be transformed into a normalized NFA $\mathcal{A}^{\mathrm{norm}}$ that recognizes $\mathsf{L}(\mathcal{A}) \setminus \{\varepsilon\}$.



Normalizing a NFA

Theorem

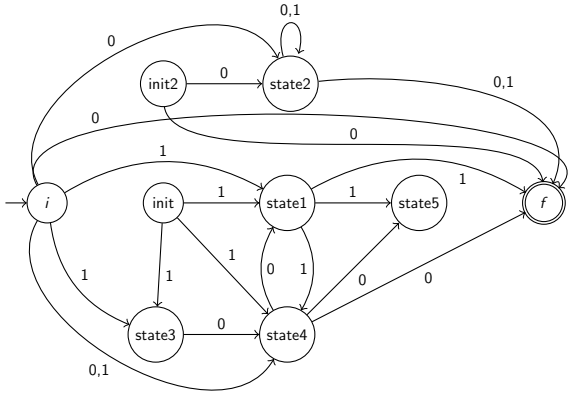
Any NFA $\mathcal{A}$ can be transformed into a normalized NFA $\mathcal{A}^{\mathrm{norm}}$ that recognizes $\mathsf{L}(\mathcal{A}) \setminus \{\varepsilon\}$.



Normalizing a NFA

Theorem

Any NFA $\mathcal{A}$ can be transformed into a normalized NFA $\mathcal{A}^{\mathrm{norm}}$ that recognizes $\mathsf{L}(\mathcal{A}) \setminus \{\varepsilon\}$.



What to do with ε?

Careful: $\mathcal{A}^{\mathrm{norm}}$ never accepts ε, even when $\mathcal{A}$ does.

What to do with ε?

Careful: $\mathcal{A}^{\mathrm{norm}}$ never accepts ε, even when $\mathcal{A}$ does. So if we had to normalize $\mathcal{A}_{r_1}$ (resp. $\mathcal{A}_{r_2}$) and $\varepsilon \in \mathsf{L}(r_1)$ (resp. $\varepsilon \in \mathsf{L}(r_2)$), the glued automaton misses the words of $\mathsf{L}(r_2)$ (resp. $\mathsf{L}(r_1)$) on their own.

What to do with ε?

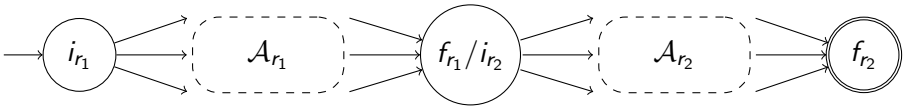
Careful: $\mathcal{A}^{\mathrm{norm}}$ never accepts ε, even when $\mathcal{A}$ does. So if we had to normalize $\mathcal{A}_{r_1}$ (resp. $\mathcal{A}_{r_2}$) and $\varepsilon \in \mathrm{L}(r_1)$ (resp. $\varepsilon \in \mathrm{L}(r_2)$), the glued automaton misses the words of $\mathrm{L}(r_2)$ (resp. $\mathrm{L}(r_1)$) on their own.

Fix: make the glue state initial when $\varepsilon \in \mathrm{L}(r_1)$, and final when $\varepsilon \in \mathrm{L}(r_2)$.

What to do with ε?

Careful: $\mathcal{A}^{\mathrm{norm}}$ never accepts ε, even when $\mathcal{A}$ does. So if we had to normalize $\mathcal{A}_{r_1}$ (resp. $\mathcal{A}_{r_2}$) and $\varepsilon \in \mathrm{L}(r_1)$ (resp. $\varepsilon \in \mathrm{L}(r_2)$), the glued automaton misses the words of $\mathrm{L}(r_2)$ (resp. $\mathrm{L}(r_1)$) on their own.

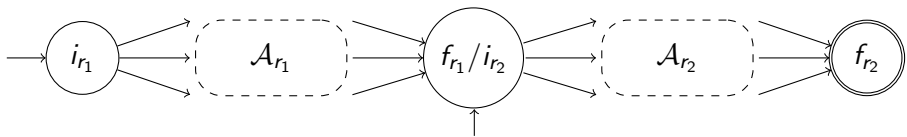
Fix: make the glue state initial when $\varepsilon \in \mathrm{L}(r_1)$, and final when $\varepsilon \in \mathrm{L}(r_2)$.



What to do with ε?

Careful: $\mathcal{A}^{\mathrm{norm}}$ never accepts ε, even when $\mathcal{A}$ does. So if we had to normalize $\mathcal{A}_{r_1}$ (resp. $\mathcal{A}_{r_2}$) and $\varepsilon \in \mathrm{L}(r_1)$ (resp. $\varepsilon \in \mathrm{L}(r_2)$), the glued automaton misses the words of $\mathrm{L}(r_2)$ (resp. $\mathrm{L}(r_1)$) on their own.

Fix: make the glue state initial when $\varepsilon \in \mathrm{L}(r_1)$, and final when $\varepsilon \in \mathrm{L}(r_2)$.

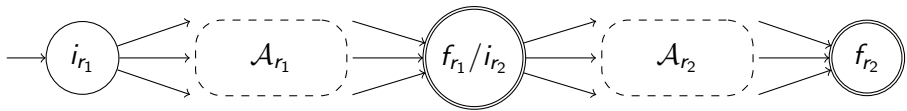


If $\varepsilon \in \mathrm{L}(r_1)$ but $\varepsilon \notin \mathrm{L}(r_2)$: words of $\mathrm{L}(r_2)$ alone must be accepted.

What to do with ε?

Careful: $\mathcal{A}^{\mathrm{norm}}$ never accepts ε, even when $\mathcal{A}$ does. So if we had to normalize $\mathcal{A}_{r_1}$ (resp. $\mathcal{A}_{r_2}$) and $\varepsilon \in \mathrm{L}(r_1)$ (resp. $\varepsilon \in \mathrm{L}(r_2)$), the glued automaton misses the words of $\mathrm{L}(r_2)$ (resp. $\mathrm{L}(r_1)$) on their own.

Fix: make the glue state initial when $\varepsilon \in \mathrm{L}(r_1)$, and final when $\varepsilon \in \mathrm{L}(r_2)$.

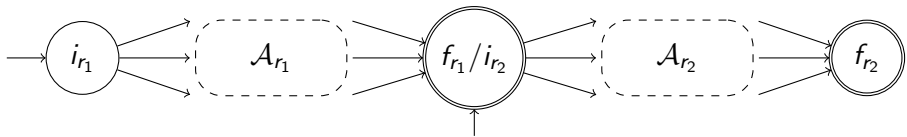


If $\varepsilon \notin \mathrm{L}(r_1)$ but $\varepsilon \in \mathrm{L}(r_2)$: words of $\mathrm{L}(r_1)$ alone must be accepted.

What to do with ε?

Careful: $\mathcal{A}^{\mathrm{norm}}$ never accepts ε, even when $\mathcal{A}$ does. So if we had to normalize $\mathcal{A}_{r_1}$ (resp. $\mathcal{A}_{r_2}$) and $\varepsilon \in \mathrm{L}(r_1)$ (resp. $\varepsilon \in \mathrm{L}(r_2)$), the glued automaton misses the words of $\mathrm{L}(r_2)$ (resp. $\mathrm{L}(r_1)$) on their own.

Fix: make the glue state initial when $\varepsilon \in \mathrm{L}(r_1)$, and final when $\varepsilon \in \mathrm{L}(r_2)$.



If $\varepsilon \in \mathrm{L}(r_1)$ and $\varepsilon \in \mathrm{L}(r_2)$: both, and ε itself.

RE to NFA: concatenation (summary)

To construct $\mathcal{A}_r$ for $r = r_1 \cdot r_2$:

RE to NFA: concatenation (summary)

To construct $\mathcal{A}_r$ for $r = r_1 \cdot r_2$:

1. Normalize $\mathcal{A}_{r_1}$ and/or $\mathcal{A}_{r_2}$ *only if* gluing is not safe.

RE to NFA: concatenation (summary)

To construct $\mathcal{A}_r$ for $r = r_1 \cdot r_2$:

1. Normalize $\mathcal{A}_{r_1}$ and/or $\mathcal{A}_{r_2}$ *only if* gluing is not safe.
2. Glue the final state of $\mathcal{A}_{r_1}$ onto the initial state of $\mathcal{A}_{r_2}$, keeping the marks of both.

RE to NFA: concatenation (summary)

To construct $\mathcal{A}_r$ for $r = r_1 \cdot r_2$:

1. Normalize $\mathcal{A}_{r_1}$ and/or $\mathcal{A}_{r_2}$ *only if* gluing is not safe.
2. Glue the final state of $\mathcal{A}_{r_1}$ onto the initial state of $\mathcal{A}_{r_2}$, keeping the marks of both.
3. If you normalized $\mathcal{A}_{r_1}$ and $\varepsilon \in \mathrm{L}(r_1)$, make the glue state initial.
   If you normalized $\mathcal{A}_{r_2}$ and $\varepsilon \in \mathrm{L}(r_2)$, make the glue state final.

RE to NFA: concatenation (summary)

To construct $\mathcal{A}_r$ for $r = r_1 \cdot r_2$:

1. Normalize $\mathcal{A}_{r_1}$ and/or $\mathcal{A}_{r_2}$ *only if* gluing is not safe.
2. Glue the final state of $\mathcal{A}_{r_1}$ onto the initial state of $\mathcal{A}_{r_2}$, keeping the marks of both.
3. If you normalized $\mathcal{A}_{r_1}$ and $\varepsilon \in \mathrm{L}(r_1)$, make the glue state initial.
   If you normalized $\mathcal{A}_{r_2}$ and $\varepsilon \in \mathrm{L}(r_2)$, make the glue state final.

Without normalization, step 3 is automatic: f_{r_1} is the only final state of $\mathcal{A}_{r_1}$, so $\varepsilon \in \mathrm{L}(r_1)$ exactly when f_{r_1} is initial.

RE to NFA: concatenation (summary)

To construct $\mathcal{A}_r$ for $r = r_1 \cdot r_2$:

1. Normalize $\mathcal{A}_{r_1}$ and/or $\mathcal{A}_{r_2}$ *only if* gluing is not safe.
2. Glue the final state of $\mathcal{A}_{r_1}$ onto the initial state of $\mathcal{A}_{r_2}$, keeping the marks of both.
3. If you normalized $\mathcal{A}_{r_1}$ and $\varepsilon \in \mathrm{L}(r_1)$, make the glue state initial.
   If you normalized $\mathcal{A}_{r_2}$ and $\varepsilon \in \mathrm{L}(r_2)$, make the glue state final.

Without normalization, step 3 is automatic: f_{r_1} is the only final state of $\mathcal{A}_{r_1}$, so $\varepsilon \in \mathrm{L}(r_1)$ exactly when f_{r_1} is initial. Same for i_{r_2}.

RE to NFA: Kleene star

Suppose $r = {r_1}^*$ and we have already constructed $\mathcal{A}_{r_1}$.

RE to NFA: Kleene star

Suppose $r = {r_1}^*$ and we have already constructed $\mathcal{A}_{r_1}$.

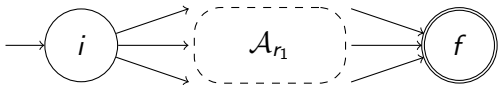
Conceptually: want to be able to loop back from any final state of $\mathcal{A}_{r_1}$ back to its initial states.

RE to NFA: Kleene star

Suppose $r = r_1{}^*$ and we have already constructed $\mathcal{A}_{r_1}$.

Conceptually: want to be able to loop back from any final state of $\mathcal{A}_{r_1}$ back to its initial states.

Put $\mathcal{A}_{r_1}$ in its normalized form (nothing to do if it already is).

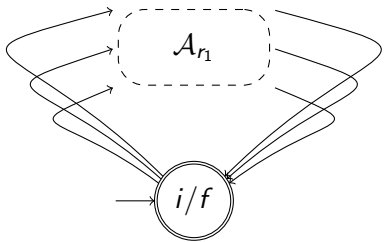


RE to NFA: Kleene star

Suppose $r = {r_1}^*$ and we have already constructed $\mathcal{A}_{r_1}$.

Conceptually: want to be able to loop back from any final state of $\mathcal{A}_{r_1}$ back to its initial states.

Put $\mathcal{A}_{r_1}$ in its normalized form (nothing to do if it already is). Then merge its initial and final states.

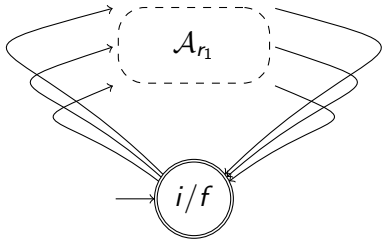


RE to NFA: Kleene star

Suppose $r = {r_1}^*$ and we have already constructed $\mathcal{A}_{r_1}$.

Conceptually: want to be able to loop back from any final state of $\mathcal{A}_{r_1}$ back to its initial states.

Put $\mathcal{A}_{r_1}$ in its normalized form (nothing to do if it already is). Then merge its initial and final states.



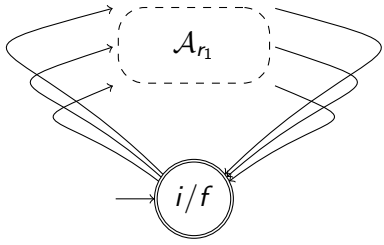
Here normalization is really needed: a transition into i or out of f would become an accepted loop at i/f that is not in $\mathrm{L}(r_1)^*$.

RE to NFA: Kleene star

Suppose $r = r_1{}^*$ and we have already constructed $\mathcal{A}_{r_1}$.

Conceptually: want to be able to loop back from any final state of $\mathcal{A}_{r_1}$ back to its initial states.

Put $\mathcal{A}_{r_1}$ in its normalized form (nothing to do if it already is). Then merge its initial and final states.



Here normalization is really needed: a transition into i or out of f would become an accepted loop at i/f that is not in $\mathrm{L}(r_1)^*$. Losing ε is harmless: $L^* = (L \setminus \{\varepsilon\})^*$.

Exercise

For next time, use the algorithm we just saw to construct an NFA recognizing

$$aa^* + a^*(ba^*)^*$$

Exercise

For next time, use the algorithm we just saw to construct an NFA recognizing

$$aa^* + a^*(ba^*)^*$$

Compare with the NFA given last time for L_2.